INFORME TÉCNICO: PIPELINE DE INTEGRACIÓN MULTIFUENTE

M4 · Sistemes de Big Data | RA1: Integración, procesamiento y análisis de información
Alumno: Daniel García Brun | **Entorno:** Python, Pandas, Requests, BeautifulSoup, PyArrow

---

1. Introducción y Contexto de Negocio

La empresa de comercio electrónico **ShopData S.L.** requería la consolidación de un repositorio analítico único a partir de tres sistemas heterogéneos y desconectados. El desafío central radicó en diseñar un pipeline reproducible y automatizado capaz de extraer los datos desde sus orígenes primarios (sin modificación manual), resolver discrepancias de granularidad y cardinalidad, validar la consistencia contable mediante aserciones y producir un dataset final en formato columnar de alto rendimiento (Apache Parquet).

2. Caracterización de las Fuentes Disponibles

El pipeline ingesta simultáneamente tres fuentes con diferentes grados de estructuración:

Fuente	Naturaleza	Volumen	Clave / Granularidad
Fuente A: Ventas (shopdata_sales.csv)	Estructurada (CSV delimitado)	75.000 transacciones (Ene-Ago 2026)	sale_id (Unívoca) N transacciones por producto
Fuente B: Catálogo (API REST DummyJSON)	Semiestructurada (JSON paginado)	Catálogo dinámico (194 productos)	id / product_id 1 fila por producto
Fuente C: Competencia (competitor_prices.html)	No estructurada (HTML Web)	31 ofertas observadas (PriceScope)	product_id M ofertas por producto

3. Extracción, Paginación Dinámica y Tratamiento de Precios

- **Paginación dinámica de la API:** A diferencia de scripts estáticos que asumen un número fijo de elementos, el pipeline consume el endpoint REST leyendo iterativamente el campo total devuelto por el servidor mediante bloques parametrizados (limit y skip). Se implementó adicionalmente un mecanismo de tolerancia a fallos que conmuta al respaldo local products_api_backup_subset.json si el servicio público no responde.
- **Web Scraping y Normalización de Precios:** La extracción desde el HTML de PriceScope procesó selectores article.offer-card, solventando la diversidad de sintaxis monetaria (ej. *'49,50 €'*, *'€ 47,90'*, *'49,50 EUR'*) mediante expresiones regulares que eliminan caracteres no numéricos y unifican el separador decimal a punto antes de convertir a coma flotante de doble precisión (float64).

4. Claves, Cardinalidades y Decisiones de Integración

El análisis previo a los cruces fue determinante para evitar la corrupción de datos:

- **Ventas vs. Catálogo (Cardinalidad N:1):** Cada venta corresponde a un producto, pero un producto se vende miles de veces. Se optó explícitamente por un **Left Join** tomando las ventas como base. Un *Inner Join* habría eliminado silenciosamente transacciones históricas de productos descatalogados.
- **Ventas vs. Competencia (Riesgo de Producto Cartesiano):** En el portal web existen múltiples ofertas de diferentes competidores para un mismo artículo (relación 1:N). Si se fusionara directamente la tabla de ventas con la de competencia, cada venta se multiplicaría por cada oferta rival encontrada. Para neutralizar esta duplicación no intencionada, se construyó una **tabla agregada intermedia (1:1 por producto)** calculando el precio mínimo observado, el precio medio y el conteo de competidores previo al merge final.

5. Cobertura de los Joins y Registros Huérfanos

El uso de indicator=True en las uniones permitió realizar una auditoría exhaustiva:

Concepto de Integración	Registros	Porcentaje	Diagnóstico de Calidad
Ventas con producto en Catálogo DummyJSON	70.627 filas	94,17%	Match exacto sobre 12 IDs activos
Ventas sin correspondencia en Catálogo	4.373 filas	5,83%	IDs 9001, 9002, 9003, 9004 (Descatalogados)
Ventas con datos de Competencia (PriceScope)	62.181 filas	82,91%	10 IDs con monitorización de precios
Ventas sin datos de Competencia	12.819 filas	17,09%	Artículos exclusivos o sin scraping rival
Total Transacciones Preservadas	**75.000 filas**	**100,00%**	**Cero duplicaciones / Cero pérdidas**

6. Diagrama de Arquitectura del Pipeline

Arquitectura del Pipeline de Integración Multifuente - ShopData S.L.

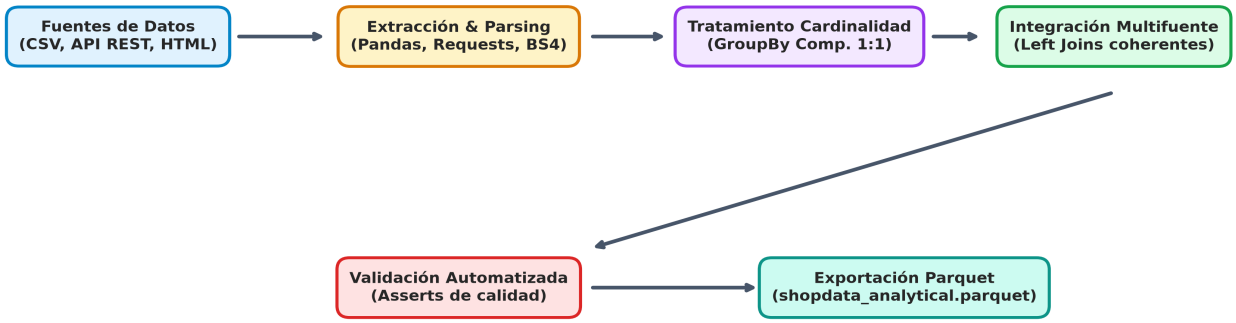


Figura 1: Arquitectura integral del pipeline desde fuentes heterogéneas hasta Parquet.

7. Principales Resultados del Análisis de Negocio

El dataset integrado df_analytical (75.000 filas y 21 columnas) permitió dar respuesta concluyente a las preguntas estratégicas de la dirección:

1. **Líderes de Facturación Histórica:** El artículo con mayor facturación es el **Watch Gold for Women (ID 193)** con **3,70 Millones de €**, seguido del **iPhone 6 (ID 122)** con **3,44 M€** y el **iPhone 5s (ID 121)** con **2,29 M€**.
2. **Categorías de Mayor Rendimiento:** El volumen de ingresos está fuertemente polarizado en smartphones (5,73 M€) y womens-watches (5,08 M€), representando conjuntamente el **72,1%** de la facturación global.
3. **Canales de Distribución:** La plataforma **Web** lidera las ventas con un 46,0% de las operaciones (6,91 M€), seguida de la **App móvil** con 34,1% (5,11 M€). Los canales directos digitales concentran más del 80% del negocio.
4. **Posicionamiento Frente a la Competencia:** El 100% de los productos monitorizados en PriceScope presentan un precio de catálogo superior al mínimo de los rivales. Las mayores desviaciones porcentuales se dan en el producto **ID 151 (+16,69%)**, **ID 31 (+14,49%)** e **ID 1 (+12,37%)**.
5. **Alerta Crítica de Rotura de Stock:** El producto estrella en ingresos históricos (**Watch Gold for Women**) se encuentra actualmente con **stock = 0** en el catálogo. Esta discontinuidad de inventario representa una fuga crítica de ventas potenciales.

8. Batería de Validación Automatizada

Para garantizar la fiabilidad del dataset en entornos de producción, el notebook incorpora aserciones programáticas que certifican de forma incondicional:

- assert df_analytical['sale_id'].is_unique: Garantiza que ninguna transacción se duplicó.
- assert len(df_analytical) == 75000: Confirma la preservación exacta del volumen transaccional original.
- assert (units > 0) & (unit_price > 0): Valida la ausencia de anomalías de signos o precios negativos.
- assert isclose(revenue, units * unit_price): Asegura la integridad contable de las variables derivadas.

9. Ventajas de la Exportación a Formato Apache Parquet

El dataset final fue serializado en shopdata_analytical.parquet mediante el motor **PyArrow**. Frente al archivo plano original (CSV), el formato Parquet ofrece ventajas determinantes en Big Data: **reducción del 37,9% en espacio de almacenamiento** (de 3,73 MB a 2,31 MB) gracias a algoritmos de compresión por bloques (Snappy), preservación estricta del esquema de tipos (eliminando la necesidad de reconvertir fechas o flotantes en cada carga) y lectura selectiva de columnas (*projection pushdown*) que acelera drásticamente las consultas analíticas en clusters distribuidos.

10. Reflexiones Técnicas y Lecciones Aprendidas

¿Por qué el código HTTP 200 no asegura la completitud del catálogo?

En servicios web orientados a gran escala, las respuestas están fragmentadas mediante paginación para mitigar sobrecargas de red. Un código HTTP 200 es solo una confirmación de éxito local sobre una única página de datos. Ignorar los parámetros de control (limit, skip, total) deriva en pipelines que ingestan subconjuntos mutilados creyendo haber finalizado con éxito.

Riesgos del Web Scraping en Entornos Productivos

A diferencia de APIs versionadas o bases de datos con contratos de esquema rígidos, el scraping depende de la estructura del Document Object Model (DOM). Cualquier refactorización de clases CSS, inserción de scripts antibot o renderizado dinámico en cliente (CSR) provoca la rotura inmediata de los extractores, obligando a mantener suites de monitorización continua sobre la integridad de las cargas.

11. Análisis de la Pregunta Final

> ***«Python ejecuta: pd.merge() sin generar ninguna excepción. ¿Significa esto que la integración es correcta?»***

No, en absoluto. La ausencia de errores de sintaxis en el intérprete de Python solo valida que las estructuras en memoria admitieron el cruce a bajo nivel, pero encubre fallos semánticos devastadores:

1. **Multiplicación Silenciosa de Filas:** Si la tabla derecha contiene cardinalidades 1:N no advertidas, Pandas genera un producto cartesiano duplicando o triplicando registros transaccionales sin emitir ninguna advertencia.
2. **Pérdida Inadvertida de Información:** Un *Inner Join* aplicado por defecto descarta silenciosamente los registros sin correspondencia, amputando datos históricos contables legítimos.
3. **Falsos Nulos por Tipos Desalineados:** Si una clave numérica fue leída como cadena de texto en un origen y como entero en otro, el merge se ejecuta limpiamente pero genera un DataFrame con todas las columnas cruzadas pobladas de NaN.

12. Conclusiones Generales

La construcción del pipeline de ShopData S.L. ha evidenciado que la integración multifuente en Big Data no consiste únicamente en conectar flujos de datos, sino en modelar rigurosamente sus cardinalidades, anticipar desajustes temporales (como productos descatalogados) y blindar la tubería con controles automáticos de calidad. El artefacto entregable satisface la totalidad de criterios de evaluación de la unidad formativa.